

SECURITY & ARCHITECTURE AUDIT REPORT

تقرير مراجعة شاملة

GOLX Sports Academy Platform – Backend API

الحالة

للاطلاع – بدون تعديل

نوع المراجعة

Static Code Analysis

تاريخ التقرير

مارس 2026

المشروع

golx-backend

CONFIDENTIAL – For Internal Use Only

تقرير مراجعة شاملة لمشروع GOLX Backend

Full Architecture & Security Audit Report

المشروع: GOLX Sports Academy Platform – Backend API تاريخ التقرير: مارس 2026 نطاق المراجعة: الكود المصدري الكامل لـ `go1x-backend/` حالة التقرير: للاطلاع فقط – لم يُجر أي تعديل على الكود

فهرس المحتويات

- نظرة عامة على المشروع
- البنية المعمارية التفصيلية
- قاعدة البيانات والمهاجرات
- البنية التحتية (Infrastructure)
- الموديولات وواجهات API
- Workers وصفوف المهام الخلفية
- الأمان – ما تم بناؤه بشكل صحيح
- نقاط الضعف والمخاطر (مفصلة)
- التوصيات مرتبة حسب الأولوية
- قائمة تحقق للتسليم

1. نظرة عامة على المشروع

الجانب	التفاصيل
الهدف	API خلفي لإدارة أكاديمية رياضية (للاعبون، مدربون، حضور، مدفوعات، ترتيبات، AI)
اللغة والبيئة	Node.js >=20, Express.js v4
قاعدة البيانات	PostgreSQL 16 عبر Knex ORM
التخزين الوسيط	Redis 7
نظام المهام الخلفية	BullMQ
التوثيق والتسجيل	Pino + pino-pretty
التحقق من المدخلات	Zod
نقطة الدخول الرئيسية	src/server.js → src/app.js
الحاويات	Docker + Docker Compose
إصدار Node المطلوب	20.0.0 =<

2. البنية المعمارية التفصيلية

2.1 نمط التصميم العام

يعتمد المشروع نمط **Layered Architecture** (طبقات متفصلة) مع **Dependency Injection** يدوي.



2.2 ملف الدخول: `src/server.js`

- يربط Redis أولاً
- يشغل BullMQ Workers
- يرفع HTTP server على المنفذ المعرف
- يدير إغلاق سلس (Graceful Shutdown) عبر SIGTERM/SIGINT
- في حالة الفشل يخرج بكود `process.exit(1)` بعد 10 ثوانٍ قسراً

2.3 ملف التطبيق: src/app.js

- يحقل جميع 9 موديولات عبر Dependency Injection يدوي (تمرير `db`, `redis`, `queue` صراحة)
- ترتيب تسجيل Middleware العالمي:
 1. `helmet()` – رؤوس أمان HTTP
 2. `cors()` – قائمة origins من `env.CORS_ORIGINS`
 3. `compression()` – ضغط الاستجابات
 4. `hpp()` – منع Parameter Pollution
 5. `cookieParser()` – تفسير الكوكيز
 6. `express.json({ limit: '2mb' })` – قراءة bodyJSON
 7. `express.urlencoded()` – قراءة form data
 8. Rate limiters على `/api/` و endpoints المصادقة
- نقطة فحص الصحة: `GET /health` – مفتوحة بدون مصادقة
- معالج 404 عام
- معالج أخطاء عالمي في النهاية

```
golx-backend/
├─ src/
│   ├─ app.js           ← DI والـ Express تهيئة
│   ├─ server.js        ← نقطة الدخول
│   ├─ config/
│   │   └─ env.js       ← التحقق من متغيرات البيئة
│   ├─ events/
│   │   └─ eventBus.js  ← EventEmitter singleton
│   ├─ infrastructure/
│   │   ├─ database.js  ← Knex + PostgreSQL
│   │   ├─ redis.js     ← ioredis client
│   │   ├─ queue.js     ← BullMQ queues
│   │   └─ storage.js   ← S3/R2 placeholder (غير مكتمل)
│   ├─ middleware/
│   │   ├─ auth.middleware.js
│   │   ├─ rbac.middleware.js
│   │   ├─ rateLimit.middleware.js
│   │   ├─ validate.middleware.js
│   │   └─ errorHandler.middleware.js
│   ├─ modules/
│   │   ├─ auth/
│   │   ├─ academy/
│   │   ├─ players/
│   │   ├─ coaches/
│   │   ├─ attendance/
│   │   ├─ rankings/
│   │   ├─ payments/
│   │   ├─ notifications/
│   │   └─ ai/
│   ├─ shared/
│   │   ├─ api-response.js
│   │   ├─ base.repository.js
│   │   ├─ base.service.js
│   │   ├─ logger.js
│   │   ├─ pagination.js
│   │   └─ errors/
```

```
| | | └─ AppError.js
| | | └─ ValidationError.js
| | | └─ UnauthorizedError.js
| | | └─ ForbiddenError.js
| | | └─ NotFoundError.js
| | | └─ ConflictError.js
| └─ workers/
|   | └─ index.js
|   | └─ ranking.worker.js
|   | └─ notification.worker.js
|   | └─ payment.worker.js
|   └─ ai.worker.js
└─ migrations/                                ← ملف مهاجرة مرقم 11
└─ Dockerfile
└─ docker-compose.yml
└─ knexfile.js
└─ package.json
```

3. قاعدة البيانات والمهاجرات

يتكون نموذج البيانات من **11 ملف مهاجرة مرقمة** تُنشئ جداول المشروع بالتسلسل:

3.1 قائمة الجداول الكاملة

رقم المهاجرة	الجدول المُنشأة	الغرض
extensions_001	(extensions + trigger)	تفعيل - uuid , pgcrypto ودالة .ossp trigger_set_updated_at()
academy_tables_002	academy_academies , academy_branches , academy_birth_years , academy_groups , academy_schedules	هيكل الأكاديمية: الفروع, سنوات الميلاد, المجموعات, الجداول
auth_tables_003	auth_users , auth_refresh_tokens , auth_password_resets	المستخدمون, توكنات التحديث, رموز استعادة كلمة المرور
players_tables_004	player_profiles , player_group_assignments , player_measurements , player_injury_history	ملفات اللاعبين, قياساتهم, إصاباتهم
coaches_tables_005	coach_profiles (وما يتبع)	ملفات المدربين
attendance_tables_006	attendance_sessions , attendance_marks	جلسات التدريب وتسجيل الحضور
rankings_tables_007	evaluation_coach_ratings , evaluation_discipline_scores , match_records , match_player_stats , ranking_snapshots , ranking_score_breakdown	التقييمات, المباريات, لقطات الترتيب
payments_tables_008	payment_subscriptions , payment_invoices , payment_transactions	الاشتراكات, الفواتير, المعاملات المالية
notifications_tables_009	notification_inbox , notification_device_tokens	صندوق الإشعارات, توكنات الأجهزة (iOS/Android/Web)

رقم المهاجرة	الجدول المنشأة	الغرض
ai_tables_010	ai_analyses , ai_recommendations , nutrition_plans	تحليلات الذكاء الاصطناعي، التوصيات، خطط التغذية
shared_tables_011	audit_logs , media_files	سجل تدقيق الأحداث، ملفات الوسائط

المجموع الكلي: ~27 جدول

3.2 ملاحظات هيكل قاعدة البيانات

- **UUID للمفاتيح الرئيسية** في كل الجداول (باستثناء `audit_logs` التي تستخدم `bigIncrements`)
- **Soft Delete**: حقل `deleted_at` في الجداول الرئيسية (`academies`, `branches`, `groups`, `players`, `coaches`, `users`)
- **Foreign Keys** بقيود `onDelete: CASCADE` أو `SET NULL` حسب السياق
- **Indexes** على الحقول المستخدمة كثيراً (`academy_id`, `user_id`, `player_id`, `status`, `created_at`)
- **Unique Constraints** على: بريد المستخدم، هاتفه، رمز التوكن، (`session_id+player_id`)، إلخ
- **Trigger** `set_updated_at` على 20 جدول لتحديث `updated_at` تلقائياً
- **Native Enums** في PostgreSQL لكل الحقول ذات القيم المحدودة (`role`, `status`, `plan`, ...)
- نوع `point` على `academy_branches.location` لتخزين الموقع الجغرافي

3.3 إعدادات الاتصال (knexfile.js)

البيئة	SSL	حجم Connection Pool
development	مُعْطَل	min:2, max:10
test	مُعْطَل	min:1, max:5
production	مُفَعَّل + <code>rejectUnauthorized: true</code>	min:2, max:10, acquireTimeout:10s

4. البنية التحتية (Infrastructure)

4.1 قاعدة البيانات (src/infrastructure/database.js)

- اتصال عبر `pg` + Knex
- SSL مشروط: مفعل في الإنتاج فقط
- Connection Pool: min:2, max:10
- اختبار اتصال عند الإقلاع عبر `SELECT 1` : فشله يُنهي العملية

4.2 (Redis (src/infrastructure/redis.js

- مكتبة `ioredis` – `lazyConnect: true`
- إعادة المحاولة: الحد الأقصى 10 مرات، زيادة تدريجية حتى 2000ms
- يُستخدم لـ: تخزين hashJI refresh tokens مؤقتاً، ودعم BullMQ
- عدم الاتصال ينهي العملية

4.3 نظام المهام (BullMQ – src/infrastructure/queue.js

تم إنشاء 4 قوائم مهام:

اسم القائمة	المنفذ (Worker)	التزامن	المهام
<code>golx-rankings</code>	<code>ranking.worker.js</code>	2	إعادة حساب الترتيبات أسبوعياً/شهرياً
<code>golx-notifications</code>	<code>notification.worker.js</code>	5	إرسال إشعار فردي أو جماعي
<code>golx-payments</code>	<code>payment.worker.js</code>	3	إنشاء فاتورة، فحص اشتراكات منتهية، معالجة استرداد
<code>golx-ai</code>	<code>ai.worker.js</code>	2	حساب أداء، تقييم خطر إصابة، خطة تغذية، AI chat

الإعدادات الافتراضية للمهام:

- المحاولات: 3 مع Exponential backoff (2000ms)
- الاحتفاظ بالمكتملة: آخر 1000

- الاحتفاظ بالفاشلة: آخر 5000

4.4 التخزين السحابي (`src/infrastructure/storage.js`)

الحالة الحالية: Placeholder كامل – كل العمليات (`upload`, `getSignedUrl`, `remove`) تُسجل تحذيراً وتعيد `null`. **الأثر:** أي ميزة رفع ملفات غير مدعومة حالياً.

4.5 نظام الأحداث (`src/events/eventBus.js`)

- `EventEmitter` مُغلّف بـ `Singleton`
- حد أقصى 50 مستمعيًا
- `publish(event, payload)`: يُصدر الحدث ويُسجله كـ `debug`
- `subscribe(event, handler)`: يلتقط الأخطاء ويُسجلها دون رمي `exception`
- مصمّم للاستبدال بـ `Kafka/RabbitMQ` مستقبلاً دون تغيير الكود

4.6 نظام التسجيل (`src/shared/logger.js`)

- مكتبة `pino` (أسرع logger في Node.js)
- `Development`: `debug` level + `pino-pretty` ملوّن
- `Production`: `info` level بدون `pretty`
- **الحساسية:** `redact` يحجب `req.headers.authorization` و `req.headers.cookie` من السجلات تلقائياً
- تسجيل context عام: `{ service: 'golx-api' }`

5. الموديولات وواجهات API

5.1 موديول (`Auth (/api/v1/auth)`)

أكثر الموديولات تعقيداً وحساسيةً.

:Endpoints

الطريقة	المسار	Auth	Rate Limit	الوصف
POST	/register	لا	authLimiter (10/15min)	تسجيل مستخدم جديد
POST	/login	لا	authLimiter	تسجيل دخول
POST	/logout	نعم	لا	تسجيل الخروج
POST	/refresh	لا	authLimiter	تجديد access token
POST	/forgot-password	لا	authLimiter	طلب إعادة تعيين كلمة المرور
POST	/reset-password	لا	authLimiter	تعيين كلمة مرور جديدة
GET	/me	نعم	لا	بيانات المستخدم الحالي

التحقق من المدخلات (Zod Schemas):

- registerSchema : بريد أو هاتف (أحدهما مطلوب). كلمة مرور 8-128 حرفاً. دور من قائمة محددة
- loginSchema : بريد أو هاتف. كلمة مرور
- forgotPasswordSchema : بريد إلكتروني فقط
- resetPasswordSchema : رمز + كلمة مرور جديدة 8-128 حرفاً

تدفق المصادقة:

```

التسجيل/الدخول → bcrypt.compare → JWT access (15m) + JWT refresh (7d)
→ refresh token الـ hash تخزين في DB
→ Redis مؤقت في hash تخزين (7d)
→ access token في body
→ refresh token في httpOnly cookie (path: /api/v1/auth)

```

تجديد التوكن (Token Rotation):

POST /refresh → cookie التحقق من الـ

→ jwt.verify REFRESH_SECRET بمفتاح

→ hash بحث عن DB في

→ إلغاء التوكن القديم

→ جديد refreshToken + جديد accessToken إصدار

استعادة كلمة المرور:

POST /forgot-password → البحث عن المستخدم بالبريد

→ token إنشأ: إنشأ (32 byte hex) عشوائي

→ مع صلاحية 1 ساعة DB في hash SHA-256 تخزين

→ eventBus عبر PASSWORD_RESET_REQ نشر حدث

→ الاستجابة دائماً نفس الرسالة (منع التعداد)

Repository الخاص بـ Auth:

- استعلامات Knex ORM مباشرة – parameterized تلقائياً
- findByEmailOrPhone مع شرط whereNull('deleted_at')
- دوال مخصصة لـ: refresh tokens, password resets, audit logs

5.2 موديول (/api/v1/academy) (Academy)

ما يدير:

- الأكاديميات (Academy)
- الفروع (Branches) مع نقاط جغرافية
- سنوات الميلاد (Birth Years)
- المجموعات (Groups) مرتبطة بسنوات ميلاد

نمط التحكم في الوصول: كل الـ endpoints تتطلب authMiddleware : عمليات التعديل/الحذف تتطلب

(rbac('*')) (admin فقط)

5.3 موديول (/api/v1/players) (Players)

ما يدير:

- ملفات اللاعبين (إنشاء، قراءة، تعديل، حذف ناعم)
- ملخص شامل للاعب (/summary)
- القياسات الجسدية (وزن، طول)

- سجل الإصابات

أذونات:

- `players:read` – للاطلاع
- `measurements:read/write` – للقياسات
- `rbac('*')` – للإنشاء والتعديل والحذف

5.4 موديول (Coaches /api/v1/coaches)

- ملفات المدربين وعملياتها الأساسية (CRUD)

5.5 موديول (Attendance /api/v1/attendance)

ما يدير:

- جلسات التدريب: إنشاء، قراءة، تحديث الحالة (scheduled → active → completed/cancelled)
- تسجيل الحضور (Batch) لجلسة كاملة
- تقرير الحضور العام

أذونات: `attendance:read/write` . `sessions:read/write`

5.6 موديول (Rankings /api/v1/rankings)

ما يدير:

- الترتيبات الأسبوعية والشهرية
- تقييمات المدربين (`evaluations`)
- المباريات وإحصاءات اللاعبين فيها
- إعادة حساب الترتيب يدوياً

خوارزمية الترتيب المخططة (في `ranking.worker`):

```
= الدرجة الكلية
%تقييم المدرب × 35
+ %نسبة الحضور × 20
+ %الانضباط × 15
+ %إحصاءات المباريات × 20
+ %درجة الذكاء الاصطناعي × 10
```

الخوارزمية موثقة في ال worker لكنها TODO حتى الآن)

5.7 موديول (Payments /api/v1/payments)

ما يدير:

- خطط الدفع (Plans)
- الاشتراكات (Subscriptions): شهري/ربع سنوي/سنوي
- الفواتير (Invoices)
- المعاملات (Transactions) مع مرجع بوابة الدفع
- تقارير المدفوعات

ملاحظة: لا يوجد تكامل فعلي مع بوابة دفع بعد – workers تحتوي TODO

5.8 موديول (Notifications /api/v1/notifications)

ما يدير:

- عرض الإشعارات وعدد غير المقروء
- إرسال إشعار فردي أو جماعي (حسب الأكاديمية والدور)
- تحديد الإشعار كمقروء (فردياً أو جماعياً)
- سجل الإشعارات المُرسلة
- توكنات الأجهزة (iOS, Android, Web) في جدول `notification_device_tokens`

ملاحظة: التكامل مع Push/Email/SMS موجود كـ TODO في الـ worker

5.9 موديول (AI /api/v1/ai)

ما يدير:

- درجات الأداء (Performance Scores)
- تقييم خطر الإصابة (Injury Risk Estimation)
- خطط التغذية المخصصة (Nutrition Plans)
- AI Chat (محادثة مع نموذج AI)

ملاحظة: كل AI workers موجودة كـ TODO (Structure جاهز، دمج النموذج الحقيقي لم يُنجز)

6. Workers وصفوف المهام الخلفية

6.1 بنية تشغيل الـ Workers

- تبدأ جميع الـ Workers في `server.js` بعد الاتصال بـ Redis
- يمكن تشغيلها بشكل مستقل عبر `npm run worker`
- إغلاق سليم عبر `stopWorkers()` الذي يستدعي `worker.close()` لكل worker

6.2 جدول الـ Workers التفصيلي

Worker	اسم القائمة	المهام	حالة التنفيذ	التزامن
ranking.worker	rankings	recalculate-rankings	TODO – هيكل جاهز فقط	2
notification.worker	notifications	deliver-notification , bulk-notification	TODO – لا يوجد تكاملي مع مزود	5
payment.worker	payments	generate-invoice , check-expiring-subscriptions , process-refund	TODO – لا يوجد تكاملي مع بوابة دفع	3
ai.worker	ai	calculate-performance , assess-injury-risk , generate-nutrition-plan , ai-chat	TODO – لا يوجد تكاملي مع نموذج AI	2

تنبيه هام: جميع الـ Workers هي هيكل جاهز فقط (Scaffolding) – لا يوجد منطق فعلي مُنفذ، كل المهام تُسجل ثم تنتهي دون فعل حقيقي.

7. الأمان – ما تم بناؤه بشكل صحيح

7.1 إدارة المتغيرات البيئية الحساسة

الملف: `src/config/env.js` – يستخدم Zod للتحقق من البيئة عند الإقلاع

المتغير	القيود	السبب
<code>JWT_SECRET</code>	≤ 32 حرفاً	لضمان entropy كافٍ
<code>JWT_REFRESH_SECRET</code>	≤ 32 حرفاً	مفتاح منفصل مختلف عن access
<code>BCRYPT_ROUNDS</code>	10-15	منع rounds منخفضة جداً أو بطيئة جداً
<code>JWT_ACCESS_EXPIRY</code>	افتراضي: 15m	نافذة تعرض قصيرة
<code>JWT_REFRESH_EXPIRY</code>	افتراضي: 7d	فقط لتجديد التوكن

إذا فشل التحقق تتوقف العملية فوراً مع رسالة واضحة.

7.2 تجزئة كلمات المرور

- مكتبة `bcrypt` المعتمدة صناعياً
- rounds من (10-15) env, لا يوجد hardcoded value
- استخدام `bcrypt.compare` للمقارنة دون إمكانية استعادة كلمة المرور

7.3 نمط JWT الآمن

- **مفتاحان منفصلان:** `JWT_SECRET` لـ access و `JWT_REFRESH_SECRET` لـ refresh
- **فترات صلاحية مناسبة:** 15 access دقيقة, 7 refresh أيام
- **لا يُخزّن التوكن كاملاً في DB** – فقط hash SHA-256
- **Token Rotation:** عند كل تحديث يُلغى القديم ويُصدر جديد
- **إلغاء شامل:** عند تغيير كلمة المرور تُلغى جميع refresh tokens للمستخدم

7.4 أمان الكوكيز

```
res.cookie('refreshToken', token, {
  httpOnly: true, // قراءتها JavaScript لا يمكن لـ
  secure: process.env.NODE_ENV === 'production', // HTTPS فقط
  sameSite: 'strict', // منع إرسالها في طلبات cross-site
  maxAge: 7 * 24 * 60 * 60 * 1000,
  path: '/api/v1/auth', // فقط auth محدودة بمسار
});
```

Rate Limiting 7.5

المحدّد	الحد	النافذة	المُطبّق على
apiLimiter	100 طلب	15 دقيقة	كل /api/
authLimiter	10 طلبات	15 دقيقة	/register , /login , /refresh , /forgot-password , /reset-password

7.6 التحقق من المدخلات

- `validate.middleware.js` يقبل Zod schemas لـ `body` , `query` , `params`
- عند فشل التحقق تُعاد بيانات مفيدة: اسم الحقل، الرسالة، المصدر
- الـ `req.body/query/params` تُستبدل ببيانات مُنقّحة من Zod

7.7 RBAC (نظام الأدوار والأذونات)

أربعة أدوار مع أذونات محددة:

الدور	الأذونات
admin	['* '] - كل شيء
coach	قراءة اللاعبين، إدارة الحضور والتقييمات والقياسات والجداول والمباريات
player	قراءة ملفه الشخصي، تقدمه، الحضور الترتيبات، القياسات، التغذية
parent	قراءة بيانات طفله، المدفوعات، الإشعارات، الحضور، الجداول

7.8 معالجة الأخطاء المركزية

يعالج: errorHandler.middleware.js

- وAصنافها الفرعية بمعلومات هيكلية `AppError`
- أخطاء DB (foreign key = 23503 = تكرار، 23505)
- أخطاء JWT
- في الإنتاج: رسالة خطأ عامة `'Internal server error'` بدلاً من رسالة الاستثناء الأصلية

7.9 رؤوس الأمان (Helmet)

يضيف تلقائياً: `helmet()`

- `Content-Security-Policy`
- `X-Frame-Options`
- `X-Content-Type-Options`
- `Strict-Transport-Security`
- وغيرها

7.10 حماية إضافية

- `hpp()` (HTTP Parameter Pollution protection) - يمنع تمرير معاملات مكررة
- `cors()` بقائمة origins صريحة من env
- `compression()` لا يؤثر على الأمان لكن يحسّن الأداء
- `redact` في logger يخفي Authorization header وcookies من السجلات

Soft Delete 7.11

تملك هذا العمود – يمنع ظهور بيانات محذوفة. يُضيف `whereNull('deleted_at')` تلقائياً لكل query عبر الجداول التي

Audit Log 7.12

جدول `audit_logs` موجود ويُستخدم في `AuthRepository.createAuditLog()` لتسجيل عمليات login مع IP والمستخدم.

8. نقاط الضعف والمخاطر (مفصلة)

● عالية الخطورة

⚠ 8.1 أسرار ثابتة (Hardcoded Credentials) في `docker-compose.yml`

الموقع: `docker-compose.yml` → السطر 10

```
POSTGRES_PASSWORD: golx_dev_pass
```

المشكلة: كلمة مرور Postgres ثابتة في ملف يُحتمل نشره في Git repository.

الخطر:

- إذا كان الريبو عاماً أو الملف مشاركاً = بيانات اعتماد DB مكشوفة

- الملف يحتوي أيضاً: `DATABASE_URL`

في `postgresql://golx:golx_dev_pass@postgres:5432/golx_main` API service

التوصية: استخدام Docker Secrets أو متغيرات بيئة خارج الريبو، وإضافة `docker-compose.yml` إلى

`.gitignore` أو استخدام `docker-compose.override.yml`

⚠ 8.2 المنافذ الداخلية مكشوفة على المضيف

الموقع: `docker-compose.yml`

```
postgres:
  ports:
    - "5432:5432" # مكشوف للعالم
redis:
  ports:
    - "6379:6379" # مكشوف للعالم
```

الخطر:

- أي شخص يعرف IP السيرفر يستطيع الوصول المباشر لـ Postgres و Redis بدون كلمة مرور (لا يوجد `requirepass` في إعدادات redis service)
- PostgreSQL محمي فقط بـ `golx_dev_pass` الثابتة

التوصية:

- حذف `ports` من postgres و redis في الإنتاج
- إضافة `requirepass` لـ Redis
- استخدام Docker networks الداخلية فقط

Redis 8.3 بدون مصادقة ⚠

الموقع: `src/infrastructure/queue.js` → redis service + `docker-compose.yml`

المشكلة: لا توجد كلمة مرور على Redis. وفي `queue.js` يُبنى الاتصال مباشرة بـ `hostname/port` فقط:

```
const redisConnection = {
  host: new URL(env.REDIS_URL).hostname,
  port: parseInt(new URL(env.REDIS_URL).port || '6379', 10),
};
```

الخطر:

- إذا كان البورت 6379 مكشوفاً = أي شخص يقدر يقرأ/يكتب في Redis
- البيانات المخزنة: hashes لتوكنات المصادقة + بيانات الـ queues

متوسطة الخطورة ●

⚠️ 8.4 غياب حماية CSRF لل Refresh Token

الموقع: `src/modules/auth/auth.controller.js` → `_setRefreshCookie`

المشكلة: ال refresh token محفوظ في cookie. رغم `sameSite: 'strict'`، هذا لا يكفي في سيناريوهات:

- تطبيقات تعمل على نطاقات مختلفة (mobile app + web)
- إذا احتجت تغيير `sameSite` مستقبلاً

الخطر:

- هجوم CSRF قد يستغل ال cookie لإصدار access token جديد

التوصية: إضافة token CSRF لل stateful endpoints، أو اعتماد نمط Double Submit Cookie، أو تغيير إرسال ال refresh token ل Authorization header مع تحقق إضافي.

⚠️ 8.5 لا يوجد إبطال فوري ل Access Tokens

الموقع: `src/middleware/auth.middleware.js`

المشكلة: `auth.middleware.js` يتحقق من التوقيع والصلاحية فقط – لا يستعلم من DB أو Redis.

الخطر:

- إذا سُرق access token، يظل فعالاً حتى انتهاء صلاحيته (15 دقيقة)
- تسجيل الخروج لا يُبطل access tokens – فقط refresh tokens
- إذا اكتُشف اختراق، لا توجد طريقة لإبطال access token للمستخدم فوراً

التوصية: إضافة (JWT ID) `jti` في ال payload، وتخزينه في Redis مع الصلاحية، والتحقق منه في

`auth.middleware.js`.

⚠️ 8.6 نشر Reset Token عبر EventBus

الموقع: `src/modules/auth/auth.service.js`

```
eventBus.publish(AUTH_EVENTS.PASSWORD_RESET_REQ, {
  userId: user.id,
  email: user.email,
  resetToken, // موجود هنا (hex string) التوكن الخام ←
});
```

المشكلة: `eventBus.publish()` يُسجّل ال payload كاملاً ك `debug` في ال logger:

```
// في eventBus.js:
logger.debug({ event, payload }, `Event published: ${event}`);
```

الخطر:

- إذا كان `log level = debug` في بيئة يَظَلَع عليها أشخاص، ال `resetToken` موجود في السجلات
- أي مستمع على ال `eventBus` يتلقى التوكن الخام كاملاً

التوصية: أرسل فقط `userId` في ال `event`. واسمح لل `notification worker` بإنشاء رابط الاستعادة من DB باستخدام `userId`.

⚠ 8.7 غياب تحقق من ملكية الموارد (Authorization Bypass Risk)

الموقع: عدة موديولات (`players, coaches, academy`)

المشكلة: RBAC تتحقق فقط من الدور، لكن لا يوجد تحقق صريح من أن المستخدم ينتمي لنفس الأكاديمية التي يُحاول الوصول لبياناتها.

مثال:

- مدرب في أكاديمية A قد يُرسل طلب `GET /api/v1/players/:id` لاعب من أكاديمية B
- إذا كان ال `service` لا يُصَفِّي على `req.user.academyId = academy_id`، تُعاد البيانات

الخطر: تسرب بيانات بين أكاديميات مختلفة (Insecure Direct Object Reference – IDOR)

التوصية: كل `service` يجب أن يُقَيِّد الاستعلام بـ `academy_id` مأخوذاً من `req.user.academyId`.

● منخفضة/متوسطة الخطورة

⚠ 8.8 Workers جميعها TODO – لم تُنَقِّذ

الموقع: `src/workers/*.worker.js`

المشكلة: ال 4 (`rankings, notifications, payments, ai`) `workers` لا تؤدي أي عمل فعلي. المهام تُسَجَّل وتنتهي.

الخطر:

- المدفوعات لا تُعالج – الفواتير لا تُنشأ، الاشتراكات لا تنتهي
- الإشعارات لا تُرسل رغم وجودها في ال `queue`
- الذكاء الاصطناعي والترتيبات لا تعمل

التوصية: يجب تحديد ال workers الحرجة (خاصة notifications payments) وإنجازها قبل الإنتاج.

Multi-stage Dockerfile بدون مرحلة 8.9 ⚠️

الموقع: Dockerfile

```
FROM node:20-alpine AS base
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/server.js"]
```

المشكلة:

- `COPY . .` ينسخ كل شيء بما في ذلك ملفات التطوير، `.env`، لو موجود، وال migrations
- لا يوجد `.dockerignore` – محتمل نسخ `node_modules` المحلية إن وُجدت
- لا يوجد مستخدم non-root – التطبيق يعمل ك `root` داخل الحاوية

التوصية:

- إضافة `.dockerignore` يستثني: `.git`، `node_modules`، `*.log`، `.env`
- إضافة `RUN addgroup -S app && adduser -S app -G app` ثم `USER app`

Rate Limiting 8.10 يعتمد على IP فقط ⚠️

الموقع: `src/middleware/rateLimit.middleware.js`

المشكلة: `express-rate-limit` بالإعداد الافتراضي يستخدم IP العميل. وراء Reverse Proxy (Nginx/Load Balancer)، قد يعتمد على `req.ip` التي تكون IP ال proxy.

الخطر:

- جميع المستخدمين خلف نفس ال proxy يشتركون في نفس الحد
- مهاجم خارج ال proxy قد لا يخضع للحد الصحيح

التوصية: ضبط `app.set('trust proxy', 1)` في Express إذا كان هناك Reverse Proxy.

⚠️ 8.11 غياب Helmet CSP مُخصّص

الموقع: `src/app.js`

```
app.use(helmet()); // إعدادات افتراضية فقط
```

المشكلة: `helmet()` بالإعدادات الافتراضية يُطبّق سياسة CSP عامة. لا توجد سياسة CSP مُخصّصة ومحكمة.

التوصية: تكوين CSP صريح يحدد المصادر المسموحة.

⚠️ 8.12 عدم وجود اختبارات آلية

الموقع: `package.json` → `"test": "jest --runInBand"`

المشكلة: Jest و `supertest` موجودان كـ `devDependencies` لكن لا يوجد دليل `/tests` أو ملفات `*.test.js` مرئية.

الخطر:

- لا يوجد اختبار لـ `auth flows`, `token rotation` و `security scenarios`
- أي تعديل مستقبلي قد يكسر سلامة الأمان دون اكتشاف

التوصية: كتابة اختبارات تكامل لـ `auth flows` الحرجة كحد أدنى.

⚠️ 8.13 `is_verified` في البيانات لكن لا يوجد آلية تحقق

الموقع: `src/modules/auth/auth.service.js` → `_sanitizeUser()`

```
isVerified: user.is_verified,
```

المشكلة: حقل `is_verified` موجود في البيانات المُعادة لكن لا يوجد في migration `auth_users` ولا توجد آلية إرسال رسالة تحقق أو التحقق من الحقل قبل السماح بالدخول.

الخطر: إمكانية تسجيل مستخدمين بأي بريد/هاتف دون تحقق من ملكيتهم له.

⚠️ 8.14 Pagination بدون حماية كافية

الموقع: `src/shared/pagination.js`

```
if (limit > env.MAX_PAGE_LIMIT) limit = env.MAX_PAGE_LIMIT;
```

الملاحظة: الحد الأقصى الافتراضي `MAX_PAGE_LIMIT = 100` – مقبول لكن يجب مراقبة الاستعلامات القوية.

⚠️ **8.15 endpoint healthcheck** مكشوف بلا مصادقة

الموقع: `src/app.js`

```
app.get('/health', (req, res) => {  
  res.json({ status: 'ok', timestamp: new Date().toISOString() });  
});
```

الملاحظة: لا يكشف بيانات حساسة (فقط `status + timestamp`). مقبول كـ `liveness probe` لكن يجب التأكد من أنه لا يكشف معلومات DB أو Redis.

9. التوصيات مرتبةً حسب الأولوية

الأولوية الأولى – عاجل (قبل الإنتاج / فوراً)

#	الإجراء	الملف المعني
1	حذف كلمة المرور الثابتة من <code>docker-compose.yml</code> وإدارة الأسرار خارج الريبو	<code>docker-compose.yml</code>
2	إزالة <code>ports</code> لـ <code>redis</code> و <code>postgres</code> أو تقييدها لـ <code>127.0.0.1</code>	<code>docker-compose.yml</code>
3	إضافة كلمة مرور لـ <code>requirepass</code> (Redis)	<code>docker-compose.yml</code> + <code>src/infrastructure/redis.js</code>
4	إضافة <code>.dockerignore</code> , يستثني <code>.env</code> , <code>*.log</code> , <code>.git</code>	إنشاء ملف جديد
5	تشغيل التطبيق بمستخدم <code>non-root</code> في <code>Dockerfile</code>	<code>Dockerfile</code>
6	التحقق من أن <code>CORS_ORIGINS</code> في الإنتاج محدد بـ <code>domain</code> حقيقي	<code>src/config/env.js</code>

الأولوية الثانية – مهم جداً (أسبوعان)

#	الإجراء
7	إزالة <code>resetToken</code> من <code>payload eventBus</code> وإرسال <code>userId</code> فقط
8	إضافة تحقق من ملكية الموارد (<code>academy_id</code>) في كل <code>service</code>
9	تنفيذ إبطال فوري للـ <code>access tokens</code> عبر <code>Redis blacklist</code> + <code>JTI</code>
10	تنفيذ آلية تحقق من البريد/الهاتف (<code>is_verified</code>)
11	إضافة <code>trust proxy</code> إعداد إذا كان هناك <code>Nginx/Load Balancer</code>

الأولوية الثالثة – تحسينات مهمة (شهر)

#	الإجراء
12	كتابة اختبارات Jest ل auth flows (register, login, refresh, reset)
13	تنفيذ ال workers الحرجة (ai → rankings → payments → notifications)
14	تكوين Helmet CSP مُخصَّص
15	تنفيذ التخزين السحابي (storage.js) بصلاحيات محدودة
16	وضع خطة دورية لتدوير JWT secrets
17	إضافة npm audit في CI/CD pipeline

10. قائمة تحقق للتسليم (Checklist)

أمان البنية التحتية

- ☐ لا توجد كلمات مرور ثابتة في `docker-compose.yml`
- ☐ منافذ Postgres/Redis غير مكشوفة للإنترنت العام
- ☐ Redis محمي بكلمة مرور
- ☐ `.dockerignore` تستثني `.env` وملفات حساسة
- ☐ التطبيق يعمل بمستخدم non-root في Docker

أمان المصادقة

- ☐ `JWT_SECRET` و `JWT_REFRESH_SECRET` أطول من 32 حرفاً ومعقدان
- ☐ `BCRYPT_ROUNDS` لا تقل عن 12 في الإنتاج
- ☐ Refresh token يُلغى عند تغيير كلمة المرور
- ☐ `resetToken` الخام لا يظهر في log
- ☐ آلية تحقق من البريد/الهاتف مُنفَّذة

أمان التحكم في الوصول

- كل service يُقيّد الاستعلام بـ `req.user.academyId` = `academy_id`
- جميع endpoints الحرجة محمية بـ `authMiddleware` + `rbac`
- لا يوجد endpoint يترك دون rate limiting مناسب

أمان الشبكة وال Transport

- TLS/HTTPS مُفعّل أمام التطبيق
- `secure: true` للكوكيز يعمل فعلياً في الإنتاج
- `CORS_ORIGINS` محدد بـ domains حقيقية فقط

الجودة والاستمرارية

- Workers حرجة مُنفّذة قبل الإطلاق (notifications, payments)
- اختبارات ل auth flows موجودة
- `npm audit` يشمل CI/CD pipeline
- Audit logs تُسجّل لعمليات الكتابة الحساسة

ملحق - تقرير الحزم والتبعيات

الحزمة	الإصدار في package.json	الغرض	ملاحظة
express	4.21.0^	HTTP framework	-
knex	3.1.0^	Query builder + migrations	-
pg	8.13.0^	PostgreSQL driver	-
ioredis	5.4.1^	Redis client	-
bullmq	5.15.0^	Job queues	-
zod	3.23.8^	Schema validation	-
jsonwebtoken	9.0.2^	JWT generation/verification	-
bcrypt	5.1.1^	Password hashing	-
helmet	8.0.0^	HTTP security headers	-
cors	2.8.5^	CORS middleware	-
express-rate-limit	7.4.1^	Rate limiting	-
pino	9.5.0^	Fast logger	-
pino-pretty	11.3.0^	Dev log formatter	-
uuid	10.0.0^	UUID generation	-
cookie-parser	1.4.7^	Cookie parsing	-
compression	1.7.5^	Response compression	-
hpp	0.2.3^	HTTP Parameter Pollution prevention	-
dotenv	16.4.5^	Load .env file	-

تنبيه: يُوصى بتشغيل `npm audit` للاطلاع على أي ثغرات معروفة في هذه الإصدارات.

نهاية التقرير

هذا التقرير أُعدَّ بناءً على مراجعة ثابتة (static analysis) للكود المصدري فقط ولم يُجرَ أي اختبار اختراق ديناميكي. تُنصح بإجراء penetration testing قبل الإطلاق الإنتاجي.